# When Repair Breaks the Interface

## A controlled test of targeted feedback for Freeway forecasts


## Abstract

We tested whether diagnosis-targeted failure examples improve executable forecast repair more than uniformly sampled failures. Forty initial programs from two Qwen sizes received two repair rounds per arm. Across 1,458,000 forecast records and 36,000 gameplay episodes, the primary targeted-minus-uniform contrast was 1.7b: +0.0 pp (95% interval +0.0 pp to +0.0 pp); 4b: +0.0 pp (95% interval +0.0 pp to +0.0 pp). This equality does not establish that targeting is ineffective: all 40 final 4B repair responses lacked the required code field, although their raw JSON contained a candidate field. Both arms therefore failed at the interface before their repaired programs could be evaluated. The smaller model retained 17 executable final programs per arm. The main finding is a failed intervention under a fixed output contract, not evidence for or against the general value of diagnosis.


## 1. Why this comparison?

Executable game models, environment feedback, and program repair are established ideas [2-5]. Real-time environments also make decision delay consequential [1]. We isolate the extra value of category-based feedback selection, keeping the initial program, prompt structure, repair limits, and controller fixed. A forecast improvement is not assumed to improve gameplay.

## 2. Design

Every initial program has three branches: original, uniformly selected failure feedback, and category-targeted failure feedback. Repairs always use the final round; unsuccessful or invalid programs remain in the analysis. The two arms share each round's sampling seed and counterbalance execution order. Each prompt contains the current program and current feedback, without accumulated chat.

Training, development validation, and test board identities are separate. Targeting ranks predefined behavioral categories by training error rate. It prioritizes failures from the worst category and fills remaining slots from other failures. Both arms receive fewer than six examples when too few failures exist. With no failures, the prompt supplies no failing example. Actual costs are reported separately.

The split holds out boards, not primitive motion inputs: 11,810 of 12,150 test car-forecast inputs (97.2%) also occur in training. The tuple is (x, timer, p, k). Supplied rules and familiar inputs make this a narrow repair test, not evidence of unseen-dynamics generalization.

# 3. Held-out forecast results

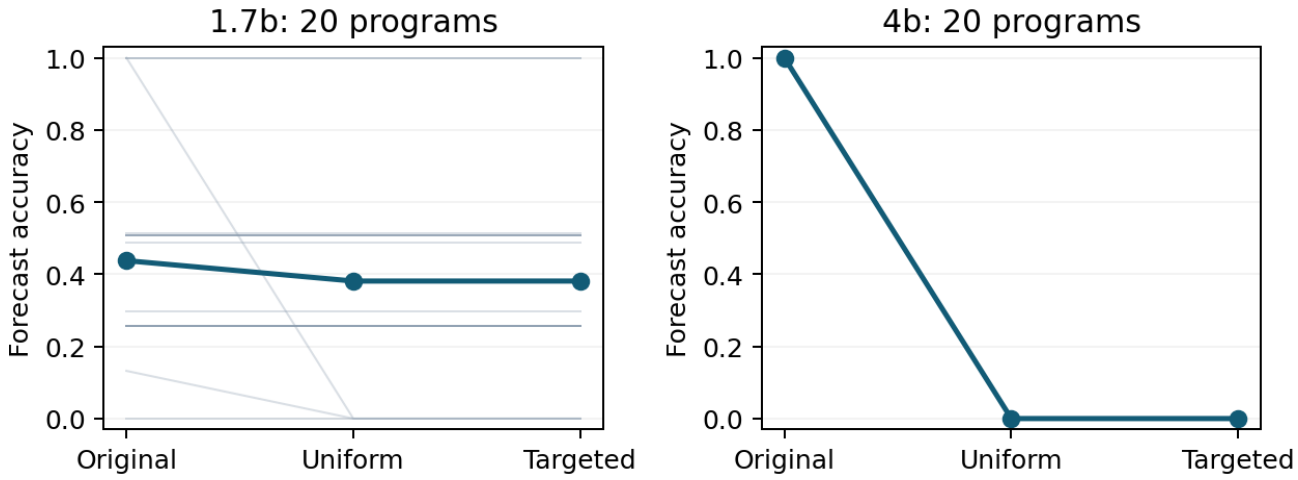



Figure 1. Each thin line is one initial-program family; the thick line is the mean. All three branches share test cases. Thousands of cases do not count as thousands of independent model generations.

| Model | Targeted - uniform | 95% interval | Holm p |
|---|---|---|---|
| 1.7b | +0.0 pp | +0.0 pp to +0.0 pp | 1.0000 |
| 4b | +0.0 pp | +0.0 pp to +0.0 pp | 1.0000 |

| Model / branch | Executable finals | Candidate without code | Accuracy |
|---|---|---|---|
| 1.7b / original | 19 / 20 | 0 | 43.8% |
| 1.7b / targeted | 17 / 20 | 3 | 38.1% |
| 1.7b / uniform | 17 / 20 | 3 | 38.1% |
| 4b / original | 20 / 20 | 0 | 100.0% |
| 4b / targeted | 0 / 20 | 20 | 0.0% |
| 4b / uniform | 0 / 20 | 20 | 0.0% |

All 4B original programs were executable and achieved perfect test accuracy. Both final repair arms instead scored zero because the output contract failed. For 1.7B, executable programs fell from 19/20 originals to 17/20 in each arm. The frozen parser required code; we did not reinterpret candidate after seeing the results. Such reinterpretation would be a separate post-hoc protocol.

The primary score weights forecast cases equally and counts invalid forecasts as wrong. Identical paired scores yield degenerate zero-width intervals; these describe this sample, not population equivalence. Bootstrap intervals preserve program pairing. Sign-flip tests use Holm adjustment over the two primary model comparisons. These are comparisons of sampled programs under this protocol, not independent draws of every forecast.

Board-macro sensitivity (equal board weights) also gives 1.7b: +0.0 pp; 4b: +0.0 pp.

The archive contains 1,458,000 forecast records, including 571,050 invalid forecasts. Board-macro scoring gives each board equal weight and is a sensitivity check.

# 4. Does repair transfer to gameplay?

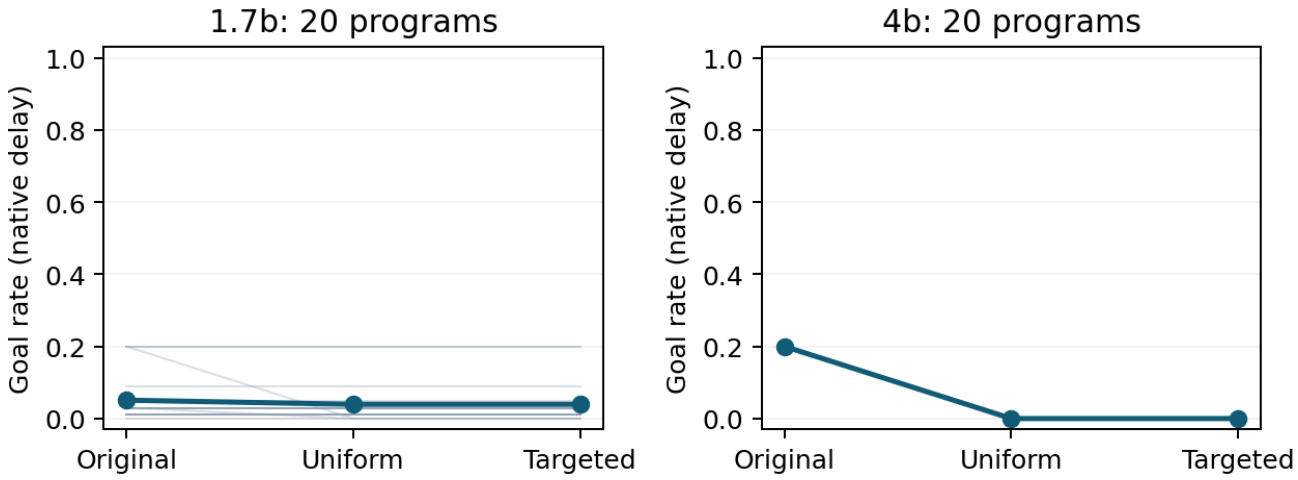



Figure 2. Goal rates under native execution delay. Lines pair branches from the same initial program; environment seeds are shared.

| Model / delay | Targeted - uniform | 95% interval |
|---|---|---|
| 1.7b / native | +0.0 pp | +0.0 pp to +0.0 pp |
| 1.7b / two_ticks | +0.0 pp | +0.0 pp to +0.0 pp |
| 1.7b / zero | +0.0 pp | +0.0 pp to +0.0 pp |
| 4b / native | +0.0 pp | +0.0 pp to +0.0 pp |
| 4b / two_ticks | +0.0 pp | +0.0 pp to +0.0 pp |
| 4b / zero | +0.0 pp | +0.0 pp to +0.0 pp |

The controller compares three decision cycles, including the waiting interval before each action. Only the forecast component is model-generated. Goal intervals preserve program pairing and shared environment seeds. This is a closed-loop rollout: changing delay changes future states and decisions.

Recorded episodes: 36,000; censored: 0. Fallback was used on 864,802 of 1,877,694 recorded decisions. A zero contrast with a wide interval is inconclusive; zero observed successes do not prove zero population success probability.

# 5. Delay and preparation cost

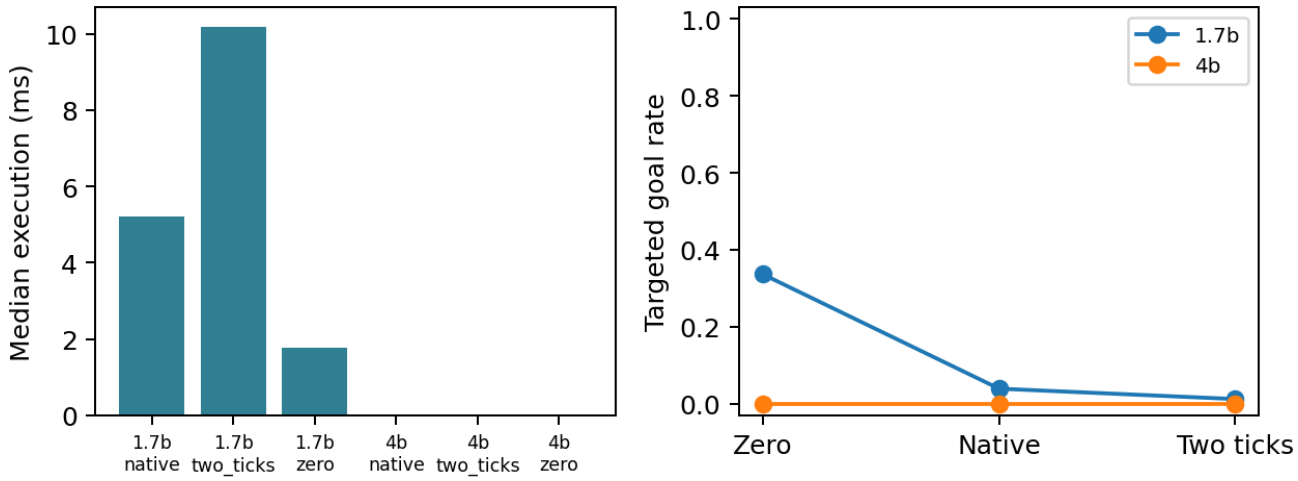



Figure 3. Measured execution time and targeted-branch goal rates across delay conditions. Injected delays are controlled stress tests, not measured language-model inference latency.

Generation and repair happen offline. Their cost is not charged anew at each game decision. Native execution is measured while task workloads are isolated. One game tick is one second; ceiling rounding charges any positive subsecond execution one tick. The controller plans one native wait tick, so executions exceeding one second are a planning-delay mismatch. Recorded mismatches: 4.

| Model / branch | Calls | Input / output tokens | Fresh / logical seconds | Cache hits |
|---|---|---|---|---|
| 1.7b / original | 20 | 4,660 / 2,588 | 47.2 / 47.2 | 0 |
| 1.7b / targeted | 40 | 21,678 / 5,591 | 92.1 / 100.6 | 4 |
| 1.7b / uniform | 40 | 21,753 / 5,413 | 87.4 / 97.9 | 5 |
| 4b / original | 20 | 4,660 / 2,355 | 101.3 / 101.3 | 0 |
| 4b / targeted | 40 | 16,960 / 4,579 | 144.8 / 189.9 | 10 |
| 4b / uniform | 40 | 16,849 / 4,577 | 143.6 / 189.4 | 10 |

Original-generation cost is shared by both repair alternatives and listed once. Repair rows contain the two additional rounds. Cached response wall times are saved generation charges, not new computation; cache counts make this visible. Equal token caps are not equal actual token use.

# 6. What feedback was selected?

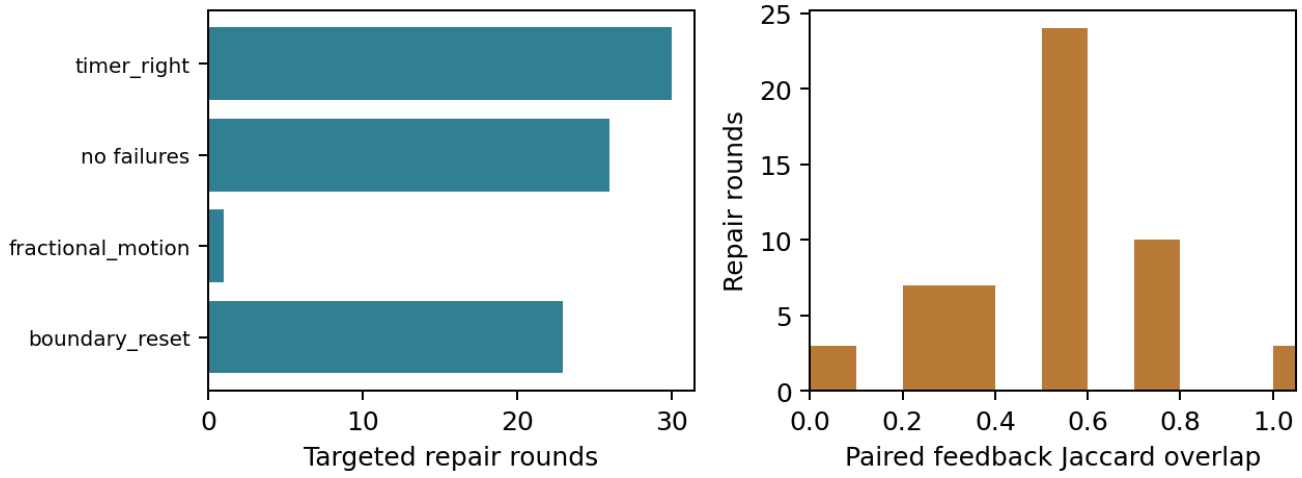



Figure 4. Targeted categories and paired feedback-set overlap. Jaccard overlap is intersection divided by union. Empty unions are unavailable and excluded from the histogram, not scored as zero.

These mechanism checks show how often the interventions actually supplied different examples. Later repair rounds can have different candidate failures because the first repairs differ. Category labels summarize externally observed errors; they do not identify an internal reasoning mechanism.

A post-hoc static audit found 1.7b: 13 distinct initial source strings among 20 requests, 17 final pairs with identical available code and 3 pairs with both sources unavailable; 4b: 1 distinct initial source strings among 20 requests, 0 final pairs with identical available code and 20 pairs with both sources unavailable. Thus the final comparison did not produce different executable treatments. Identical scores and zero-width intervals are not evidence of general equivalence.

The 200 logical requests included 29 cache hits. Repeated cached responses are not independent fresh model calls. Alternative candidate fields were inspected only for text and grammar; they were not executed or rescored. The frozen primary results remain unchanged.

# 7. Discussion and limits

Neither model showed an observed targeted-versus-uniform advantage. For 4B, that contrast is blocked by a shared output-interface failure: every final repair was rejected before execution. The intended semantic repair comparison was therefore not realized for that model. The 1.7B arms also had identical paired scores and lost executable programs relative to their initial versions. All 4B first-round feedback sets were empty: the initial programs passed training checks, so the first repair prompt did not implement distinct targeting. The fixed two-round policy still requested revision and broke already-correct programs. Stopping early or accepting the original would be a different, unrun policy. Static inspection of candidate fields does not establish that their contents would pass if accepted. These findings support auditing interface validity before interpreting repair effectiveness. They do not identify whether feedback targeting would help under a reliable interface or a harder unsaturated forecasting task.

The controller was weak under delay even with correct forecasts in development: the ten-seed pilot achieved 8/10 goals at zero delay and 1/10 in each delayed regime. These are development checks, not matched test controls. The planned 100-seed assisted test control was not collected before the research cutoff. Gameplay is therefore a limited secondary result, not a clean ceiling-normalized test of forecast transfer. No new research runs were started after the cutoff.

Only Freeway and two sizes in one model family are tested. The generated motion function is simpler than a full world model. Most primitive test inputs also occur in training despite disjoint boards. The fixed controller can limit or mask forecast improvements. Tick rounding creates a coarse latency threshold. The initial programs, not episodes,

determine how much independent evidence we have about generation and repair variability. Earlier studies are development context and are not pooled into this experiment.

# 8. Reproducibility

The release includes prompts, candidates, correct feedback examples, split identities, raw responses, per-case outcomes, replayable trajectories, analysis code, and an evidence manifest. The independent audit must pass before this builder accepts the summary. Protocol deviations and assisted-reference results must accompany the final evidence package.

# References


[1] Wen et al. (2025). Real-Time Reasoning Agents in Evolving Environments. https://arxiv.org/abs/2511.04898

[2] Lehrach et al. (2025). Code World Models for General Game Playing. https://arxiv.org/abs/2510.04542

[3] Lou et al. (2026). AutoHarness: improving LLM agents by automatically synthesizing a code harness. https://arxiv.org/abs/2603.03329

[4] Orvalho et al. (2025). Counterexample Guided Program Repair Using Zero-Shot Learning and MaxSAT-based Fault Localization. AAAI. https://arxiv.org/abs/2502.07786

[5] Zhu et al. (2025). Where LLM Agents Fail and How They can Learn From Failures. https://arxiv.org/abs/2509.25370


# Evidence receipt

```
summary.json: 854fa60a38fe2ebfbb9c6c7358f6a3ca2e666e6432838a06e83402fae4db1fa6
forecast.jsonl.gz: 5dc8273ad32779d6edcad97a5e84a115e584da208174fd2eb98a0a5d986ec9ec
live.jsonl: b39aa879e5931b924235fd3e09ada65bf4b0c577cfcb18e0b0f3dcb7db76f761
candidates.jsonl: 29db98c8d1cf64f5f6af7beaa2c7e2686a19a77a11d569a90ebedfee5480d64c
failure-audit.json: bf6a7a9a5b9551d69378307bbb7af49fd9181f162f2274666e0ccab709a145ac
```

All 125 input hashes, including trace shards, are saved in the companion evidence JSON file.

This paper describes the new targeted-repair study only. Previous paper and evidence archives remain separate. A passed audit checks records and scoring; it is not a guarantee of scientific novelty or publication acceptance.